

Cornell Bowers C-IS
College of Computing and Information Science

Deep Learning

Week 6: Image-to-Image Models/ GANs

Thanks to:

Varsha Kishore

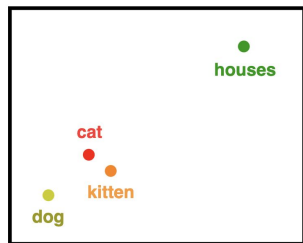
Justin Lovelace

Zachary Ross

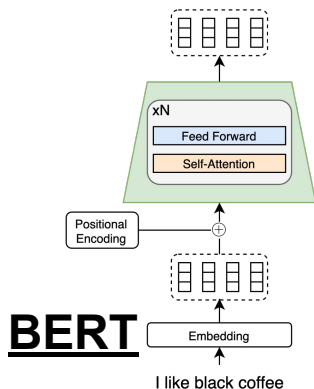
Madhav Aggarwal

Story so Far...

Understanding (Classification)

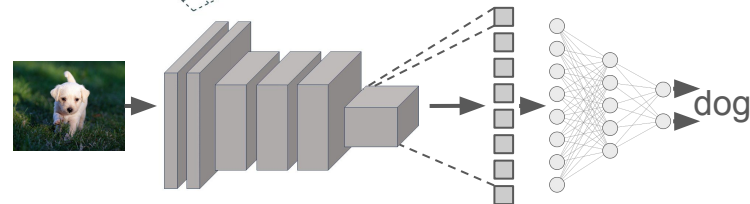


Text

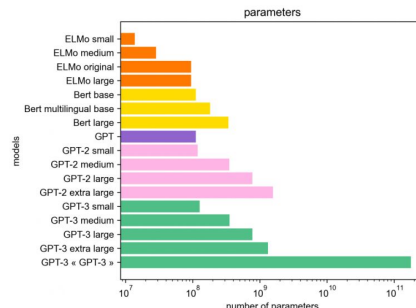
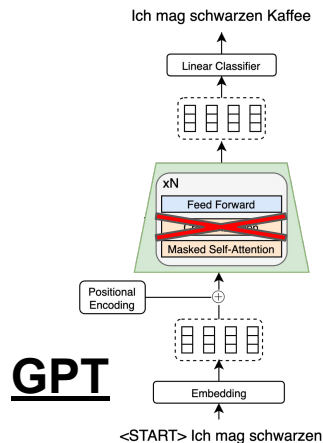


Image

Conv Nets ViTs



Generation



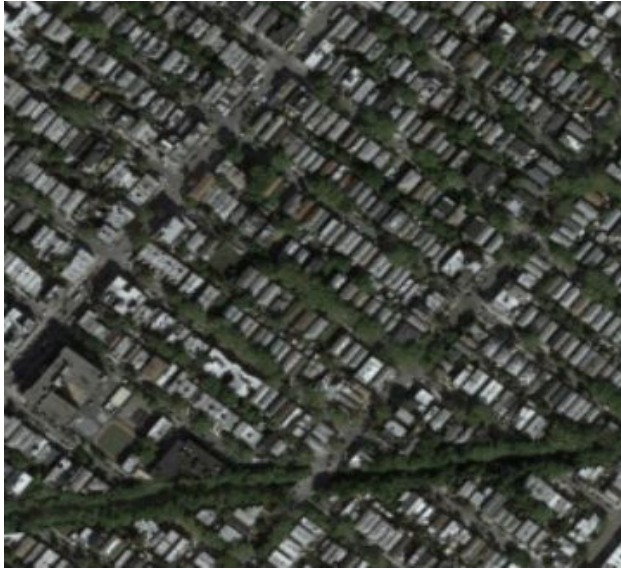
???



Overview

- Image-to-image tasks
- Image-to-image networks
- Unpaired image translation
- Generative Adversarial Networks
 - Issues & how to tackle them

What are the relationship between these two images?



For paired data, how can we train a model to...



Image Super-resolution

For paired data, how can we train a model to...

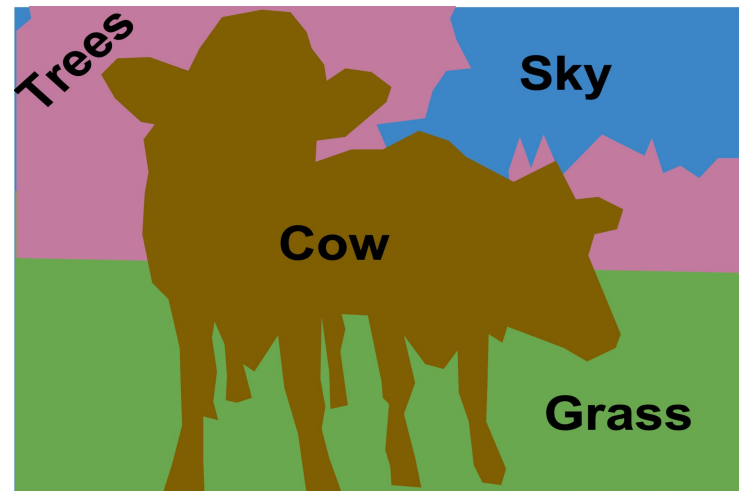


Image Segmentation (specifically, semantic segmentation)

Review: Image Classification



Input Image

Classification



Cow

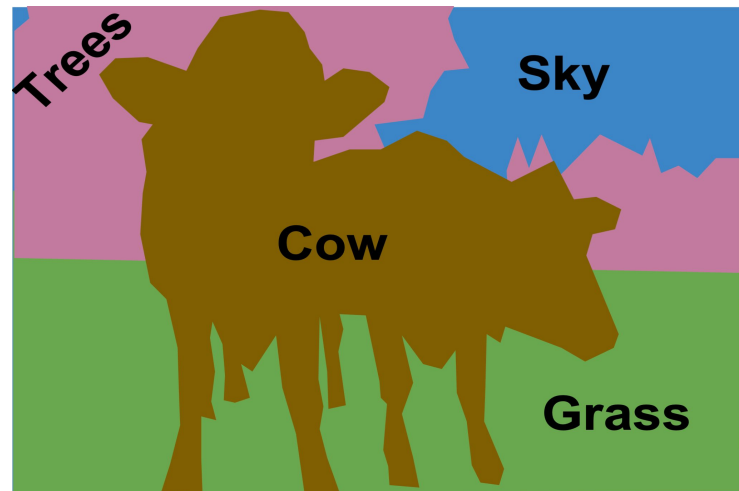
Image-level Prediction

Image-to-Image Task



Input Image

Semantic
Segmentation



Pixel-level Prediction

Applications in Autonomous Driving

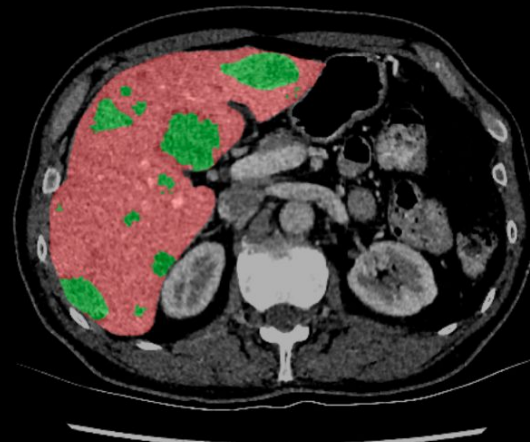
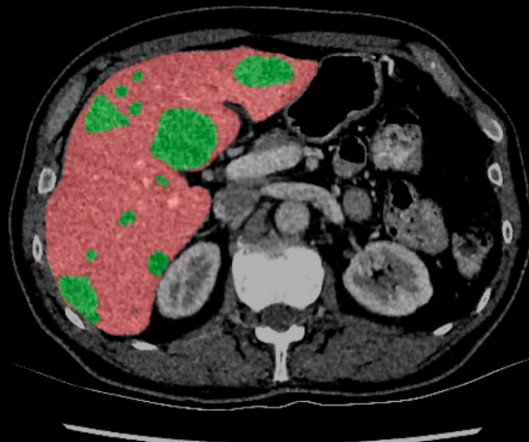


Applications in Medical Imaging

Input

Automatic segmentation

Manual segmentation



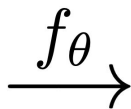
Semantic Segmentation

Task Formulation

Take an *image* of dimension $(H, W, 3)$ and output a *segmentation map* of dimension $(H, W, 1)$.

Formally, it is a function f , parameterized by θ , that produces a segmentation map of C classes.

$$f_{\theta} : \mathbb{R}^{H \times W \times 3} \longrightarrow \mathbb{N}^{H \times W \times 1}$$



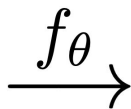
1: cow
2: grass
3: tree
4: sky



Image-to-Image Generation

A *segmentation map* of dimension $(H, W, 1)$ can be viewed as a *generated image*.

$$f_{\theta} : \mathbb{R}^{H \times W \times 3} \longrightarrow \mathbb{N}^{H \times W \times 1}$$



1: cow
2: grass
3: tree
4: sky



Image-to-Image Generation

A *segmentation map* of dimension $(H, W, 1)$ can be viewed as a *generated image*.

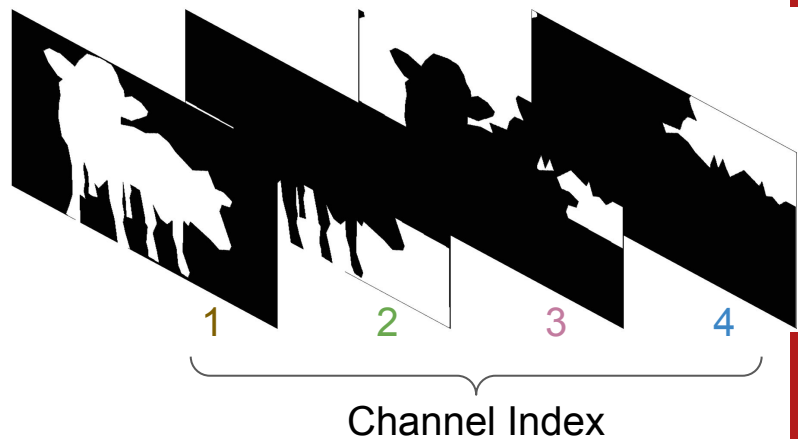
Instead of outputting integers for each pixel, the model outputs a vector of length C

$$f_{\theta} : \mathbb{R}^{H \times W \times 3} \longrightarrow \mathbb{R}^{H \times W \times C}$$



$f_{\theta} \rightarrow$

- 1: cow
- 2: grass
- 3: tree
- 4: sky



How to build image-to-image networks?

Building a Image-to-Image Network from Scratch

✓ Convolutions

Allow parallelization when extracting latent vector for each pixel

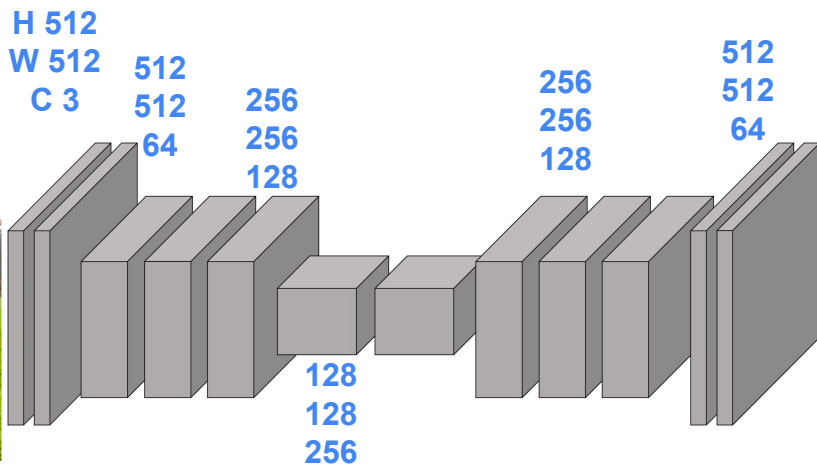
✓ Hourglass

Improve efficiency by reducing computations with downsampling

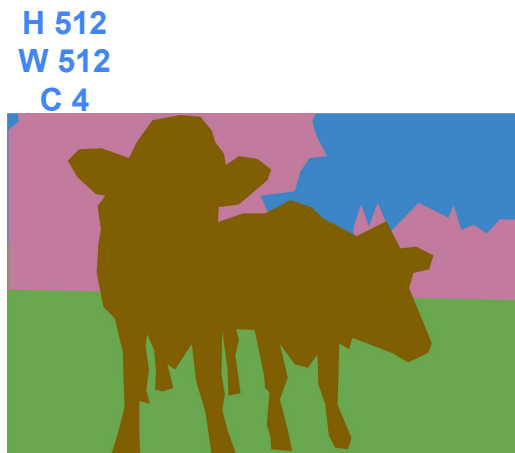
Increase receptive field size by convolving on downsampled feature maps



Input Image



Hourglass CNN



Prediction

Building a Image-to-Image Network from Scratch

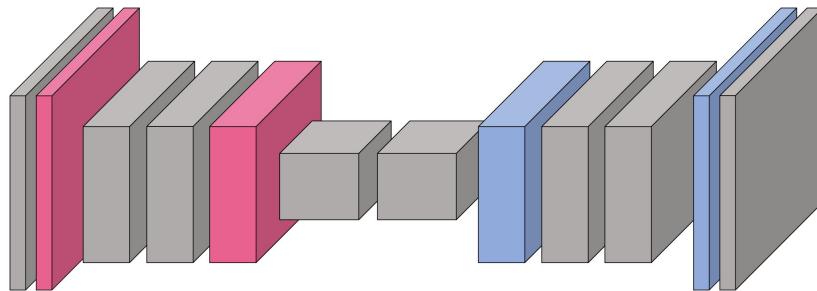
(pooling, strided
convolution)

downsample

upsample (??)



Input Image



Prediction

Upsampling - Unpooling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

"Bed of Nails"

1	2
3	4

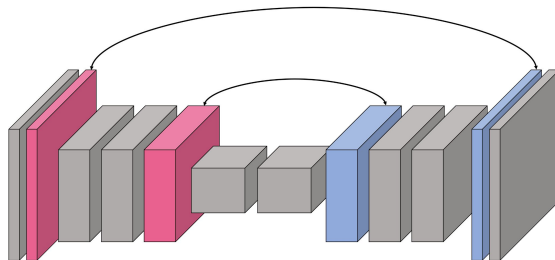


1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

Output: 4 x 4

Does not recover all spatial
information lost during
downsampling!



Upsampling - Max Unpooling

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2



Rest of the network

Max Unpooling

Use positions from pooling layer

1	2
3	4

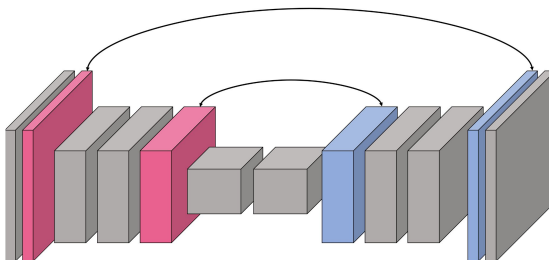
Input: 2 x 2



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

Corresponding pairs of
downsampling and
upsampling layers



Up-Convolution

Input

1	2
3	4

Filter

4	3
2	1

**

** : Up-Convolution

Output

$$\begin{bmatrix} 4 & 3 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 8 & 6 \\ 0 & 4 & 2 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 12 & 9 & 0 \\ 6 & 3 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 16 & 12 \\ 0 & 8 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 11 & 6 \\ 14 & 30 & 14 \\ 6 & 11 & 4 \end{bmatrix}$$

Building a Image-to-Image Network from Scratch

✓ Convolutions

Allow parallelization when extracting latent vector for each pixel

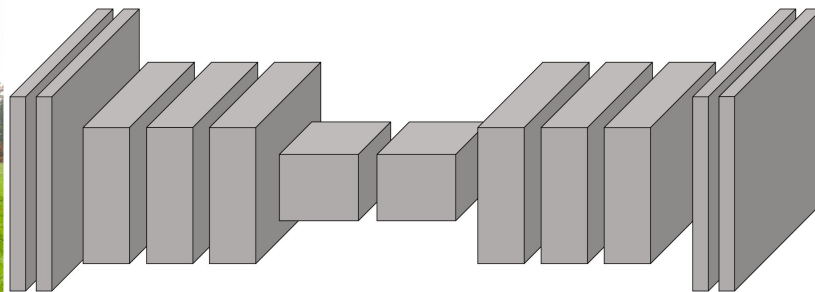
✓ Hourglass

Improve efficiency by reducing computations with downsampling

Increase receptive field size by convolving on downsampled feature maps



Input Image



Hourglass CNN



Prediction

U-Net Architecture!

“U-net: Convolutional networks for biomedical image segmentation.” Ronneberger et al., MICCAI 2015

- ✓ Convolutions
- ✓ Hourglass
- ✓ Skip Connections

Allow parallelization to extract latent vector for each pixel

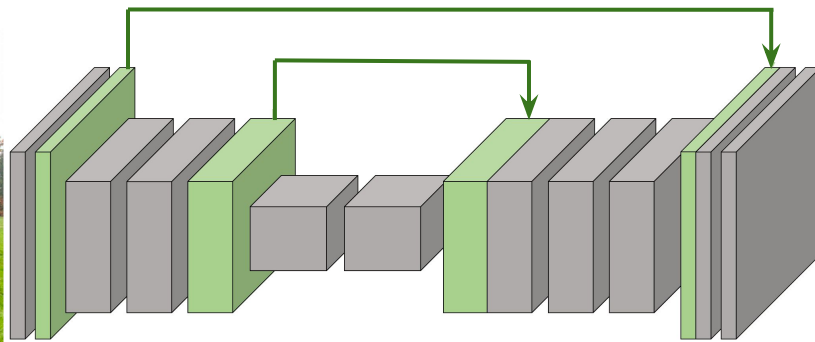
Improve efficiency by reducing computations with downsampling

Increase receptive field size by convolving on downsampled feature maps

Improve prediction quality by combining low-level image features



Input Image



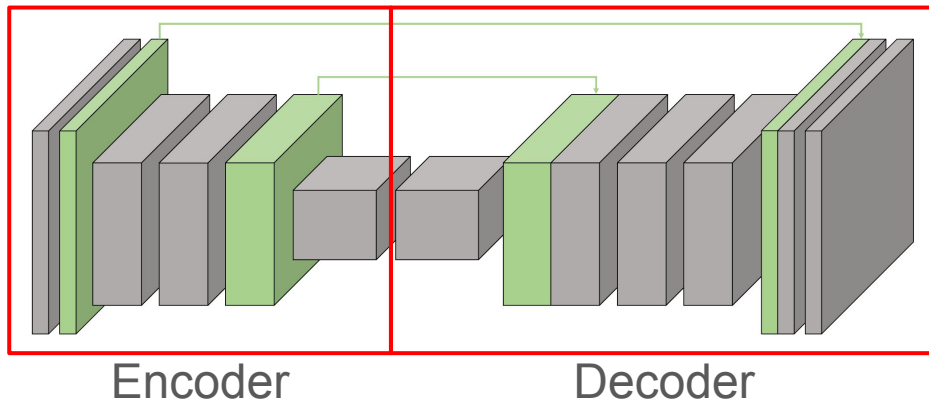
Hourglass CNN with Skip Connections

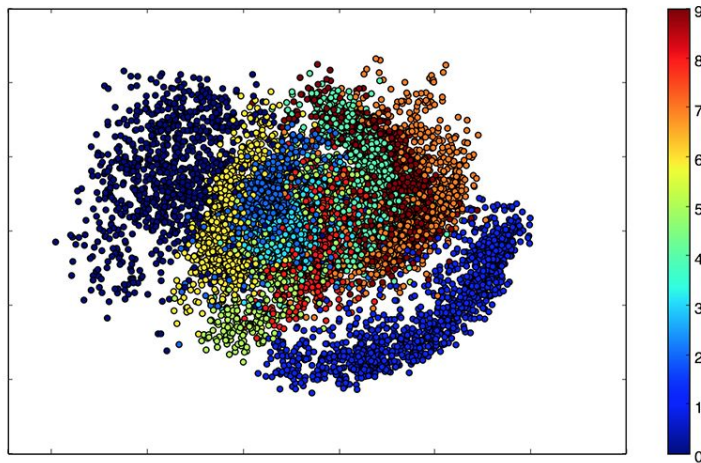
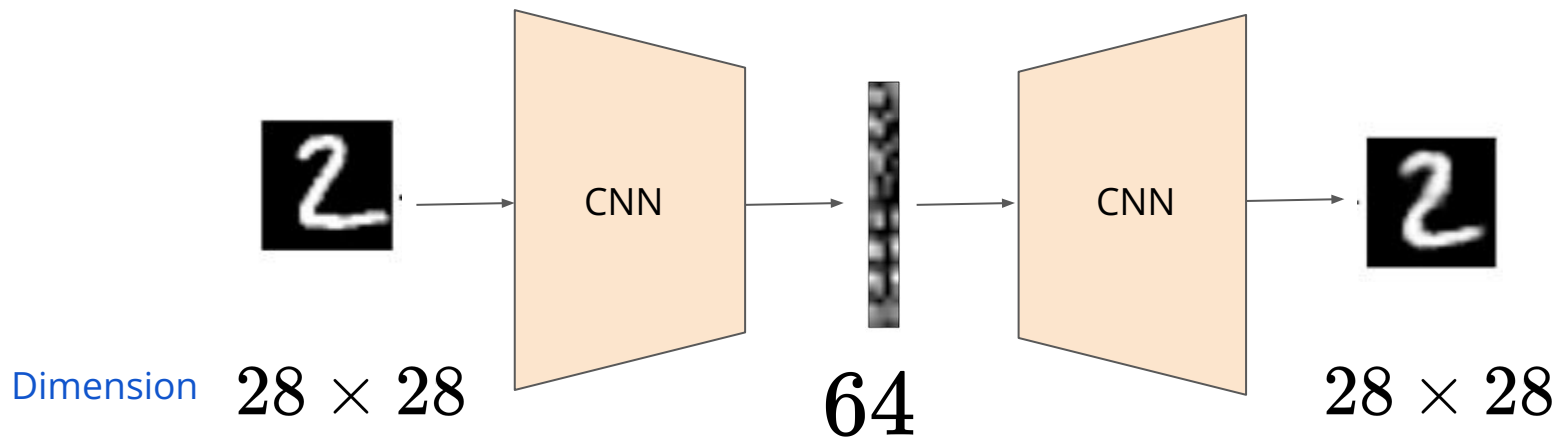


Prediction

Encoder-Decoder Perspective

- Encoder:
 - Maps an image to a low-resolution, semantically meaningful feature map
 - Basically ResNet!
- Decoder:
 - Maps a low-dimensional feature map to an image
- Can use one or the other depending on the application
 - Similar to transformers for text





Semantic Segmentation with U-Net (Supervised)

✓ Convolutions

Allow parallelization to extract latent vector for each pixel

✓ Hourglass

Improve efficiency by reducing computations with downsampling

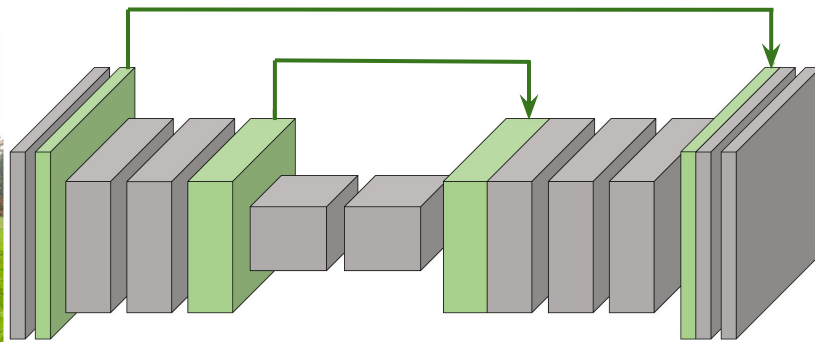
Increase receptive field size by convolving on downsampled feature maps

✓ Skip Connections

Improve prediction quality by combining low-level image features



Input Image



Hourglass CNN with Skip Connections

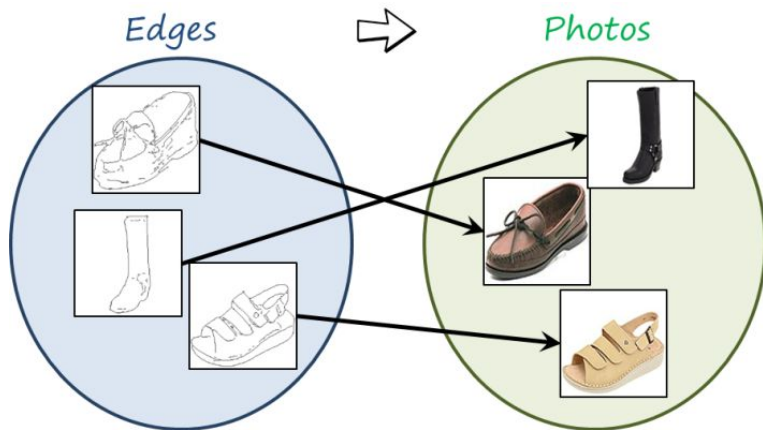


Prediction

What happens if we don't have labels?

Unpaired Image Translation

Paired



[Generative adversarial networks and image-to-image translation | Luis Herranz](#)

Paired limitations - it is hard to find exact pairings

Unpaired



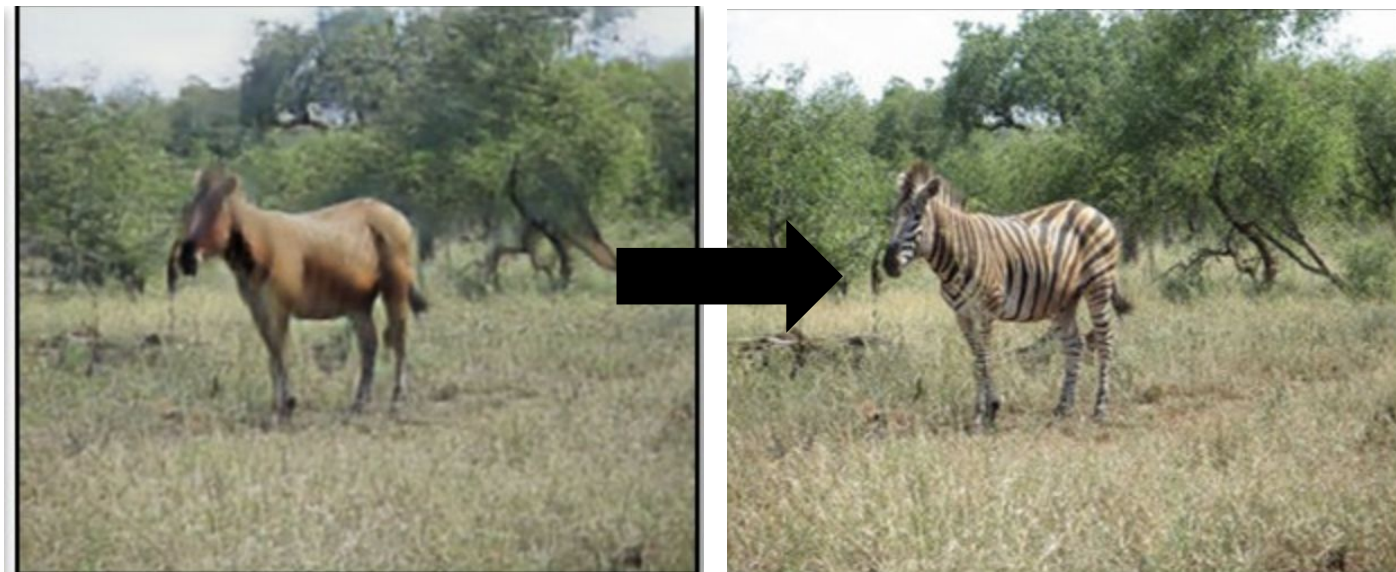
[12 Astonishing Facts About Horses](#)



[Zebra Facts | Live Science](#)

Problem with Paired Approaches with Unpaired Translation

How can we tell if we produced a good output without any reference?

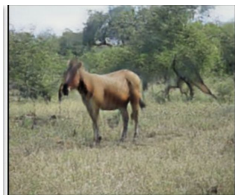


Classifier (Discriminator) to the rescue!

- A binary classifier that determines whether a given image is a zebra or not
- Can actually use as a learning signal!

Not Zebra

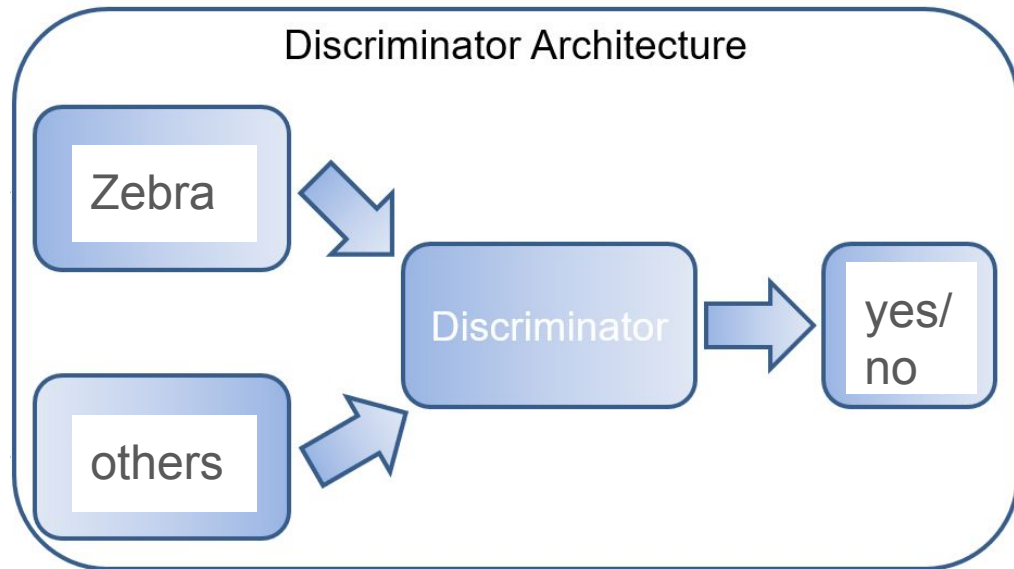
Zebra



Hyperplane

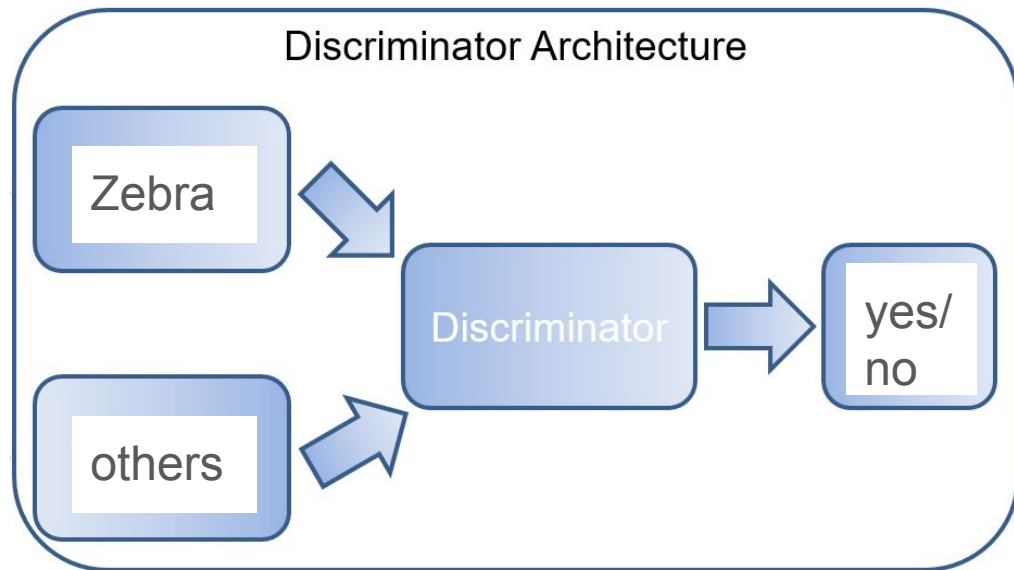
How do we get/train this discriminator?

- Supervised learning
- Train discriminator with the **binary cross-entropy loss**



How do we get/train this discriminator?

- Supervised learning
- Train discriminator with the **binary cross-entropy loss**



$$\min_D [-y \log(D(\mathbf{x})) - (1 - y) \log(1 - D(\mathbf{x}))]$$

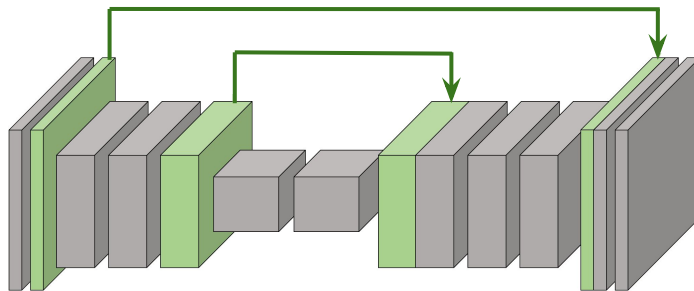
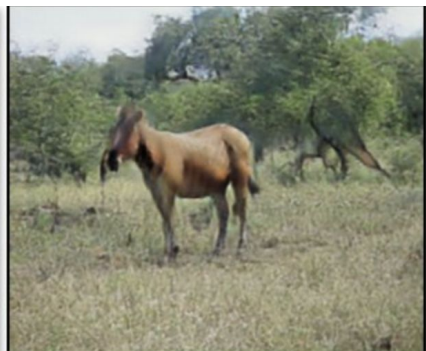
To Generate Realistic Images, We Need:

- A learning signal to make the images realistic (Discriminator)
 - Binary classification
- A way to generate images (Generator)
 - Recall image-to-image translation

Generative Adversarial Networks

Generator (U-Net)

- Can parameterize our generator as a U-Net!
- How to train?



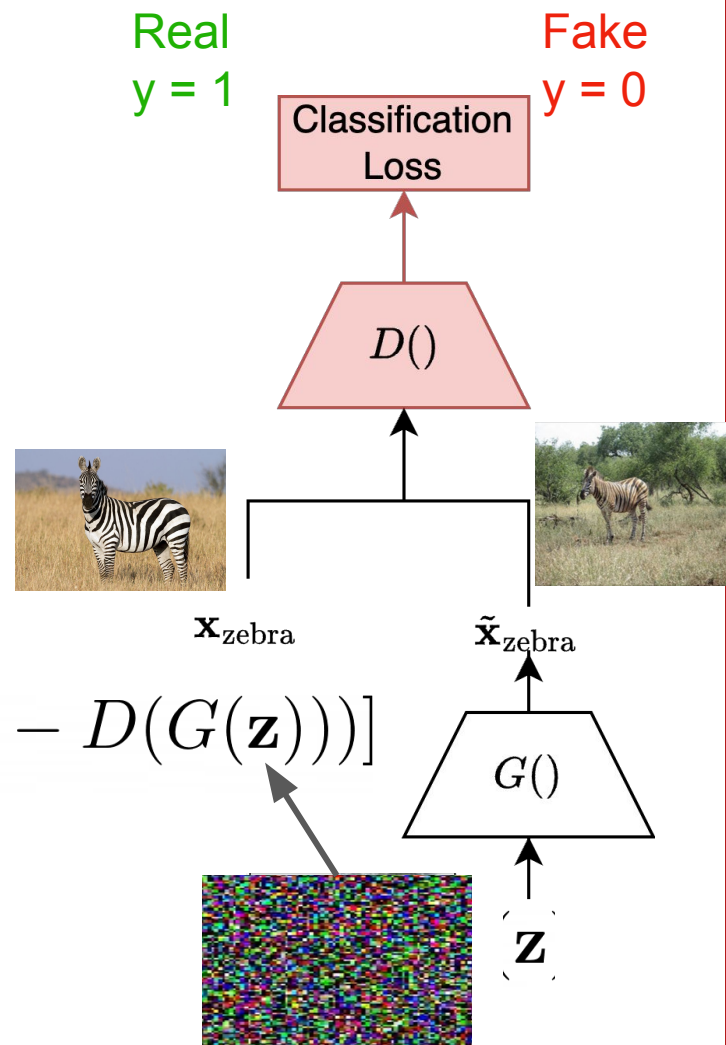
Adversarial Training

- Train the discriminator to identify real and fake zebra images
- Train the generator to fool the discriminator!
 - It should generate images that look like zebras

Goal of discriminator

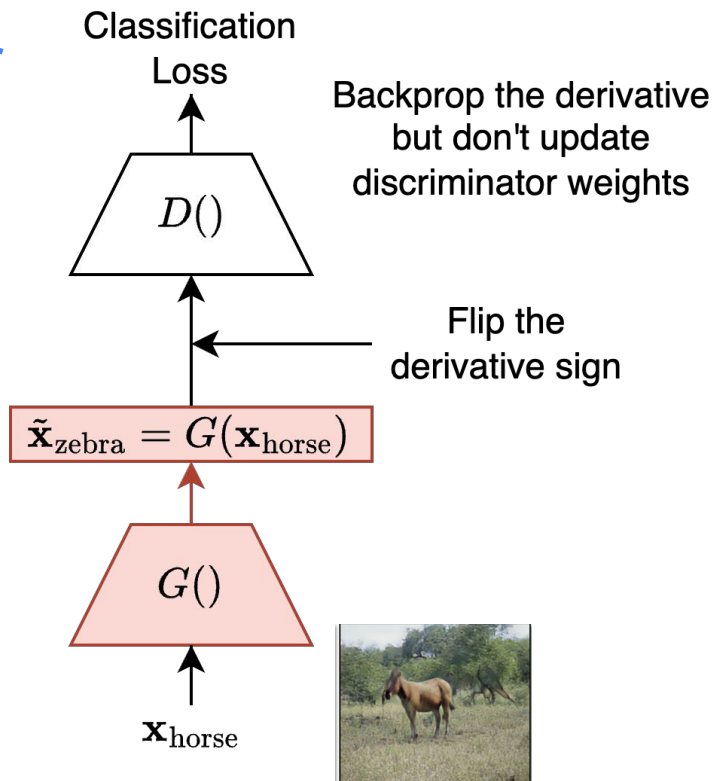
$$\max_G \min_D [-y \log(D(\mathbf{x})) - (1 - y) \log(1 - D(G(\mathbf{z})))]$$

Goal of generator



$$\max_G \min_D [-\log(D(\mathbf{x})) - \log(1 - D(G(\mathbf{z})))]$$

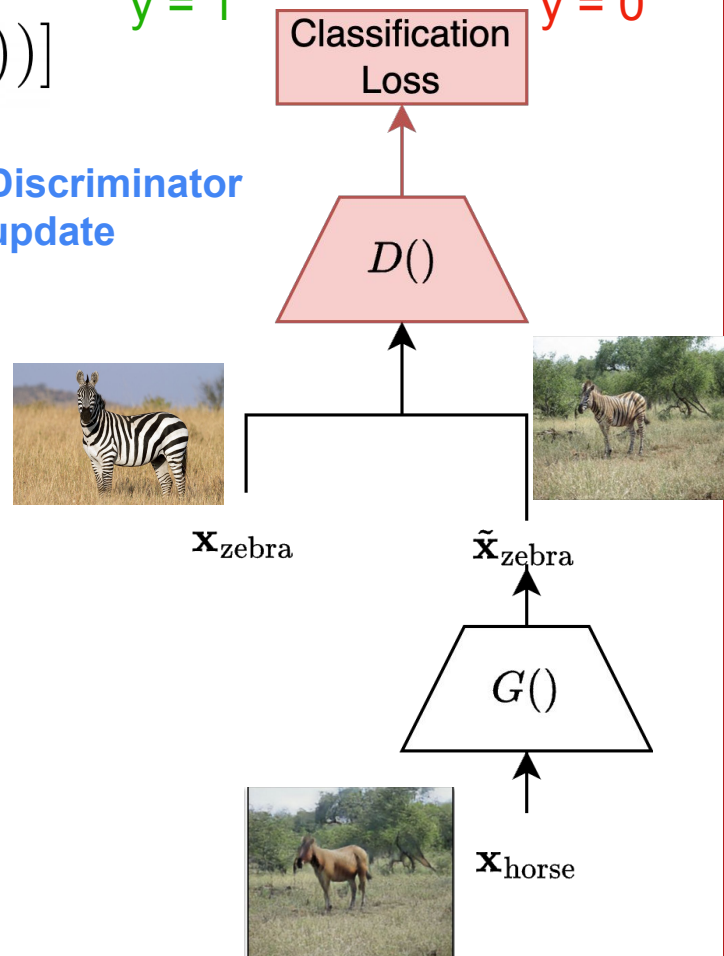
Generator
update



Real
 $y = 1$

Fake
 $y = 0$

Discriminator
update



Example

Real dogs



dog (1)



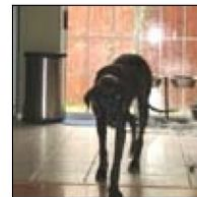
dog (1)



dog (1)

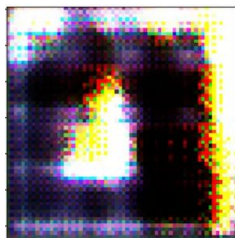


dog (1)



dog (1)

Generated



Discriminator
Loss

**Generally
Decrease**

**Ideal:
Confused**

—————▶ Training epoch

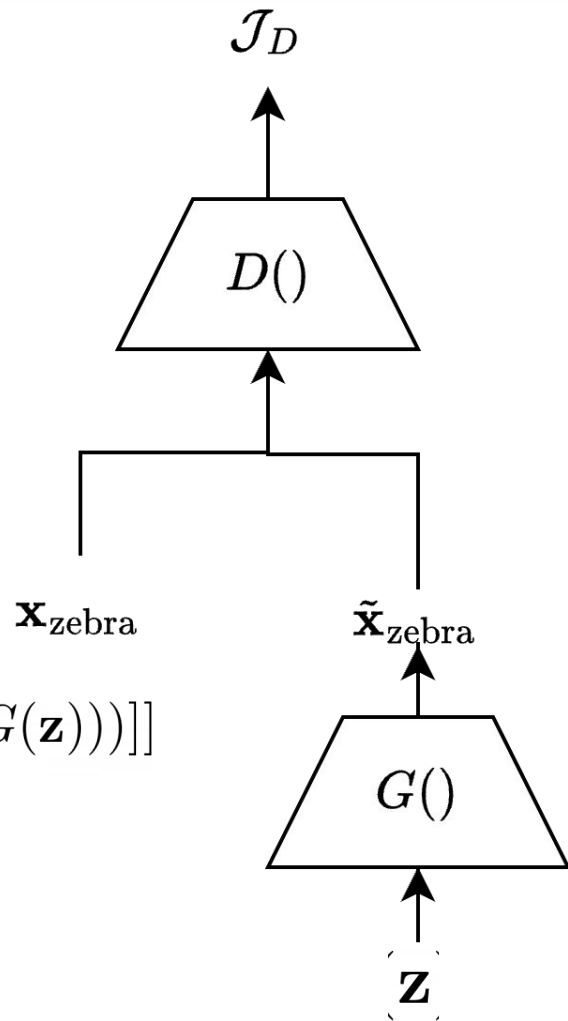
Full GAN loss

Per sample:

$$\max_G \min_D [-\log(D(\mathbf{x})) - \log(1 - D(G(\mathbf{z})))]$$

Full:

$$\max_G \min_D [\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [-\log(D(\mathbf{x}))] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [-\log(1 - D(G(\mathbf{z})))]]$$



Issue #1

Limitations of GANs

- Minimax training objective is hard to optimize!
 - Can lead to oscillations/instability during training
- Not guaranteed to converge to a good solution
 - Sensitive to hyperparameter settings, network architectures, and the choice of loss functions

Real
 $y = 1$ Fake
 $y = 0$

Generative Adversarial Networks (GANs)

- Minimax cost runs into vanishing gradient problems with a strong discriminator
 - No learning signal for the generator!

Original (minimax)

$$\max_G [-\log(1 - D(G(\mathbf{z})))]$$

$$\min_G [\log(1 - D(G(\mathbf{z})))]$$

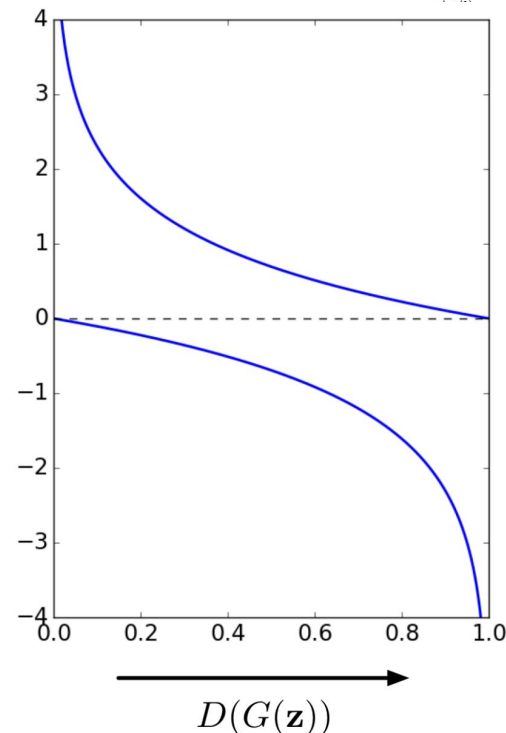
= minimize discriminator
predicting **fake**

Modified

$$\max_G [\log(D(G(\mathbf{z})))]$$

$$\min_G [-\log(D(G(\mathbf{z})))]$$

= maximize discriminator
predicting **real**

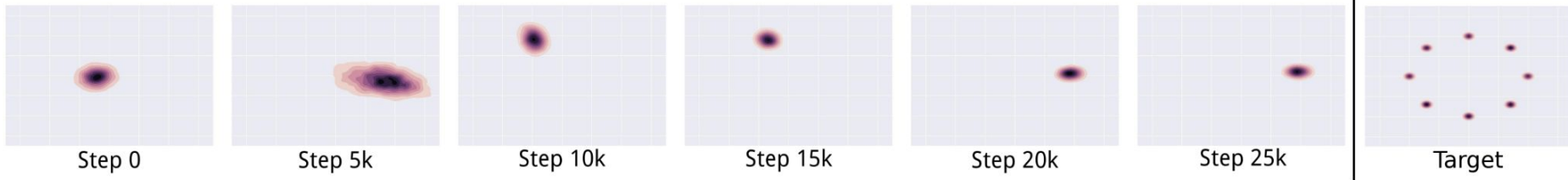
modified
costminimax
cost

(how well it fooled
the discriminator)

Issue #2

Mode Collapse

- Big problem in practice
- GANs often fail to model the full distribution of images
 - “Collapse” to some popular mode to fool the discriminator



Limitations of GANs

- Consider training a GAN on a dataset of dogs and cats
- Generator could specialize in generating realistic dogs
 - Successfully fools the discriminator!



dog (1)



dog (1)



dog (1)



cat (0)



dog (1)



dog (1)



cat (0)



cat (0)



dog (1)

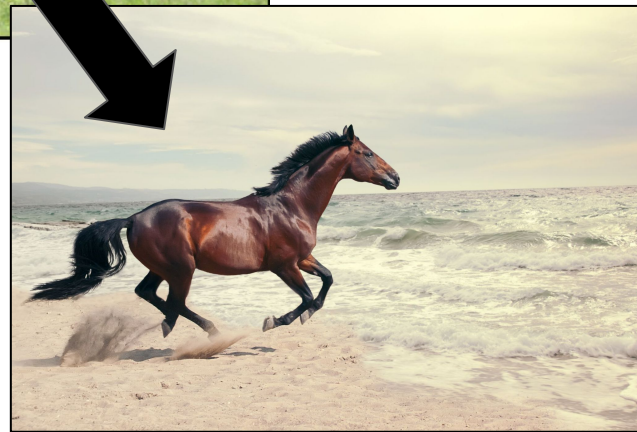
Issue #3

Unpaired Image Translation

- Want to preserve information about the original image in the generated image

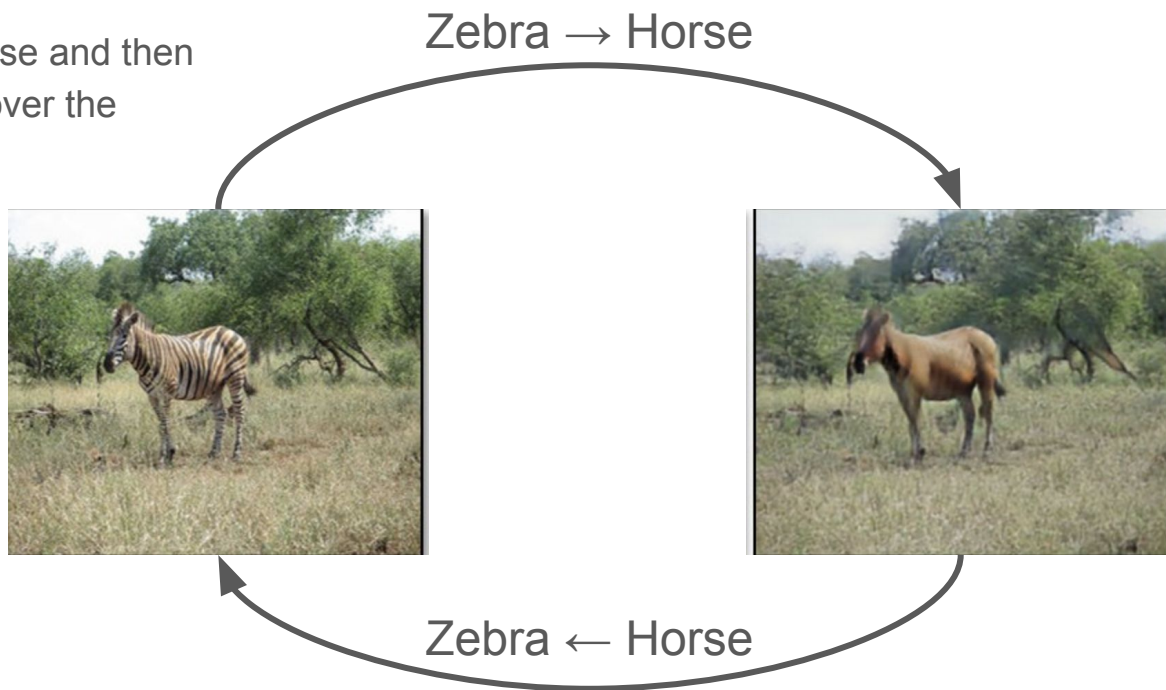


Information
not preserved



Key Idea: Cycle Consistency

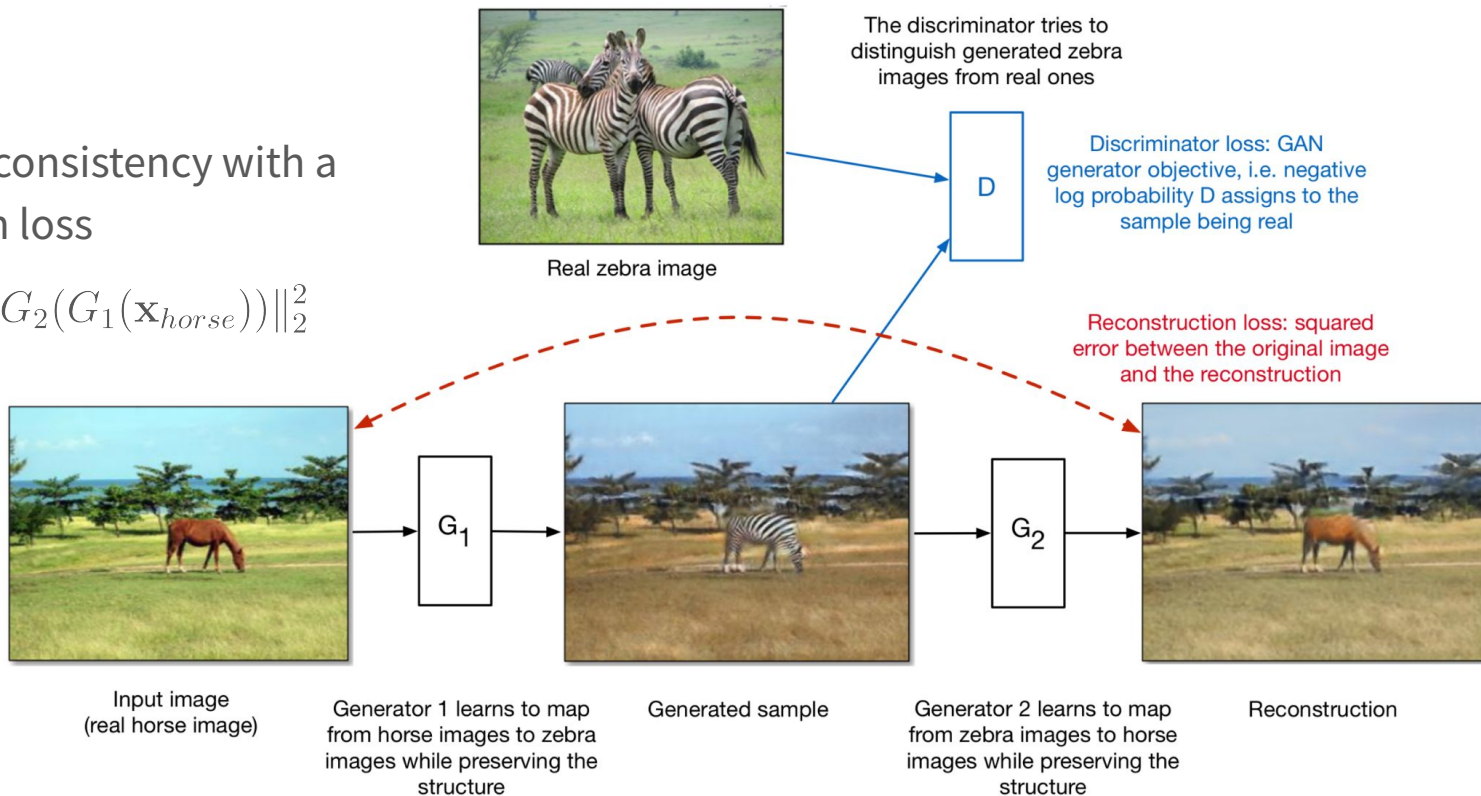
- Image translation should be **invertible!**
 - Translating a zebra to a horse and then back to a zebra should recover the original image
- Cycle consistency!



CycleGAN

- Enforce cycle consistency with a reconstruction loss

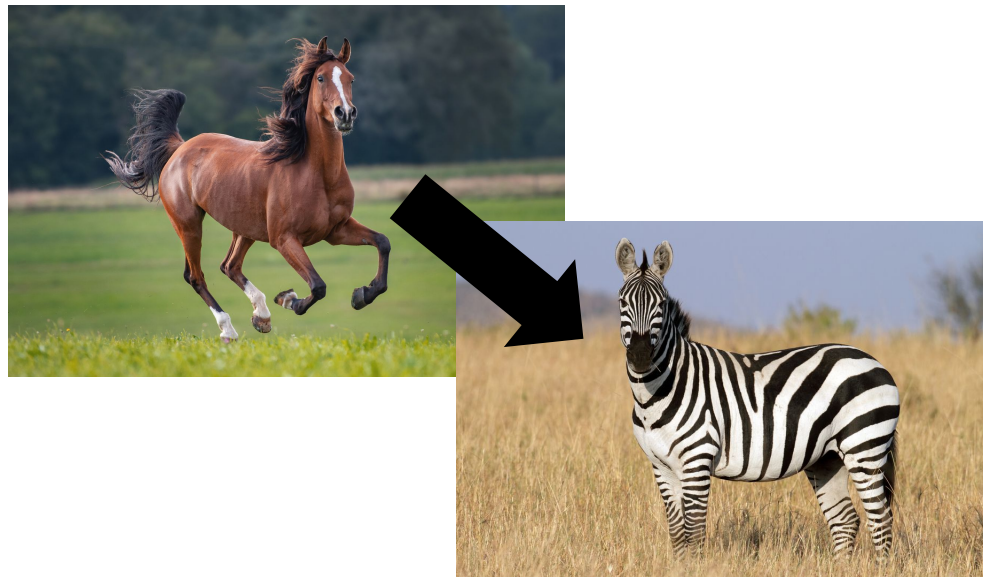
$$\|\mathbf{x}_{horse} - G_2(G_1(\mathbf{x}_{horse}))\|_2^2$$



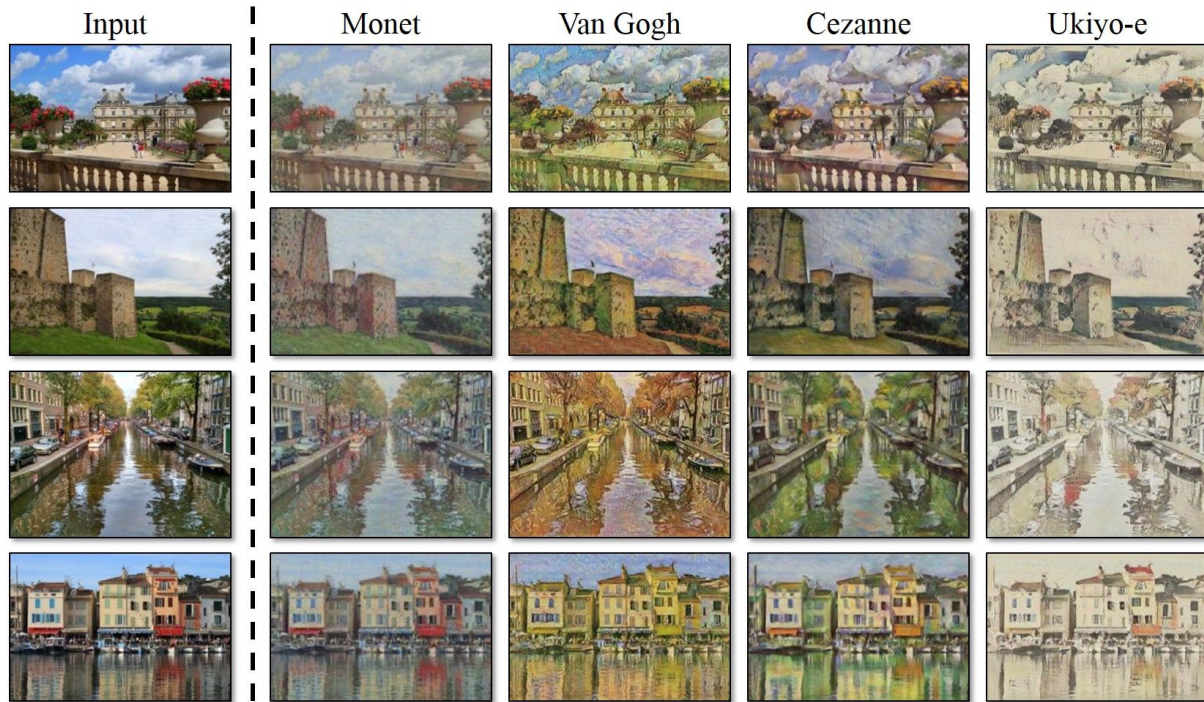
$$\text{Total loss} = \text{discriminator loss} + \text{reconstruction loss}$$

What are some applications of unpaired translation?

Unpaired

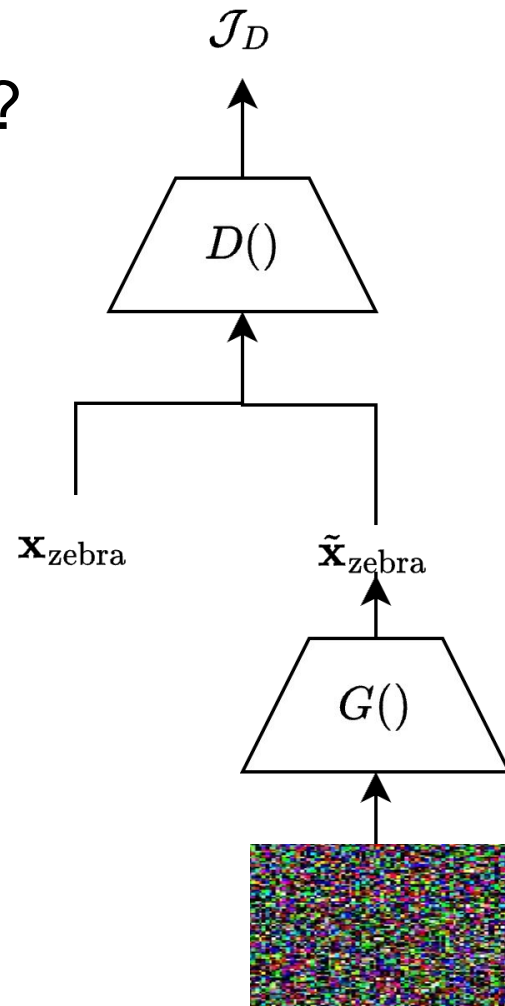


The Power of Unpaired Translation



Can we perform unconditional generation?

- Just want to draw samples from some distribution of images (e.g. zebras)
- Replace the source image with Gaussian noise



GANs

- First generative model capable of realistic high-resolution image synthesis
- Very fast generation!



Latent Space Properties

Gaussian noise vector is meaningful

- Similar noise vectors lead to similar images
- Noise dimensions are meaningful!
- Can exploit this to control generation



Recap

- Many vision tasks can be formulated as image-to-image problems
 - Segmentation, super-resolution, etc.
- The U-Net is a versatile encoder-decoder architecture for these tasks
- CycleGAN can perform unpaired image translation by learning to fool a discriminator
- GANs can perform unconditional image generation conditioned on samples of Gaussian noise
 - Challenging to train
 - Susceptible to mode collapse